

LeetCode Java Problem-Solving Guide

15 Problems • Understand → Visualize → Code

A Beginner-Friendly Problem-Solving Workbook

Table of Contents

#	Problem Name	Pattern / Topic	Difficulty
2235	Add Two Integers	Math	Easy
2469	Convert the Temperature	Math	Easy
2413	Smallest Even Multiple	Math	Easy
1281	Subtract Product and Sum of Digits	Digit Extraction	Easy
1342	Number of Steps to Reduce a Number	Simulation	Easy
412	Fizz Buzz	Simulation / String	Easy
2520	Count the Digits That Divide a Number	Digit Extraction	Easy
2652	Sum Multiples	Math	Easy
9	Palindrome Number	Digit Extraction	Easy
258	Add Digits	Math	Easy
509	Fibonacci Number	Dynamic Programming	Easy
1137	N-th Tribonacci Number	Dynamic Programming	Easy
231	Power of Two	Math / Bitwise	Easy
326	Power of Three	Math	Easy
1523	Count Odd Numbers in an Interval	Math	Easy

How to use this workbook: Read the problem carefully. Understand the pattern and visual explanation before looking at the code. Finally, trace the dry run to see exactly how the variables change during execution.

Problem 2235 — Add Two Integers

EASY

PATTERN: MATH

1. UNDERSTAND THE PROBLEM

Given: Two integers, num1 and num2.

Find: The sum of these two integers.

Example: Input: num1 = 12, num2 = 5 → Output: 17

2. KEY IDEA

DIRECT ADDITION

This is the most fundamental introductory problem. In Java, the + operator handles standard integer addition.

3. JAVA SOLUTION

```
class Solution {  
    public int sum(int num1, int num2) {  
        // Return the calculated sum directly  
        return num1 + num2;  
    }  
}
```

4. COMPLEXITY

Time: O(1)

Space: O(1)

The time and space complexities are both O(1) because we perform a single mathematical operation requiring no extra memory scaling with input size.

Problem 2469 — Convert the Temperature

EASY

PATTERN: MATH

1. UNDERSTAND THE PROBLEM

Given: A non-negative floating point number representing a temperature in Celsius.

Find: An array containing the temperature converted to Kelvin and Fahrenheit.

- $\text{Kelvin} = \text{Celsius} + 273.15$
- $\text{Fahrenheit} = \text{Celsius} * 1.80 + 32.00$

2. VISUALIZE THE PROBLEM



3. JAVA SOLUTION

```
class Solution {  
    public double[] convertTemperature(double celsius) {  
        double kelvin = celsius + 273.15;  
        double fahrenheit = celsius * 1.80 + 32.00;  
  
        // Return results in a new double array  
        return new double[]{kelvin, fahrenheit};  
    }  
}
```

4. COMPLEXITY

Problem 2413 — Smallest Even Multiple

EASY

PATTERN: MATH

1. UNDERSTAND THE PROBLEM

Given: A positive integer n .

Find: The smallest positive integer that is a multiple of both 2 and n .

2. HOW TO THINK ABOUT IT

We need a number that is divisible by both 2 and n .

- If n is already an even number (divisible by 2), then n itself is the smallest multiple of both 2 and n .
- If n is an odd number, it is not divisible by 2. To make it a multiple of 2, we must multiply it by 2. Thus, $n * 2$ is the answer.

3. JAVA SOLUTION

```
class Solution {  
    public int smallestEvenMultiple(int n) {  
        // If n is even (n % 2 == 0), return n.  
        // Otherwise, return n * 2.  
        if (n % 2 == 0) {  
            return n;  
        } else {  
            return n * 2;  
        }  
    }  
}
```

4. DRY RUN

Input (n)	Condition ($n \% 2 == 0$)	Action	Result
5	$5 \% 2 \neq 0$ (Odd)	Return $5 * 2$	10
6	$6 \% 2 == 0$ (Even)	Return 6	6

5. COMPLEXITY

Time: $O(1)$

Space: $O(1)$

Problem 1281 — Subtract the Product and Sum of Digits of an Integer

EASY

PATTERN: DIGIT EXTRACTION

1. UNDERSTAND THE PROBLEM

Given: An integer n .

Find: The difference between the product of its digits and the sum of its digits.

Example: $n = 234$

Product = $2 * 3 * 4 = 24$

Sum = $2 + 3 + 4 = 9$

Result = $24 - 9 = 15$

2. PATTERN: DIGIT EXTRACTION

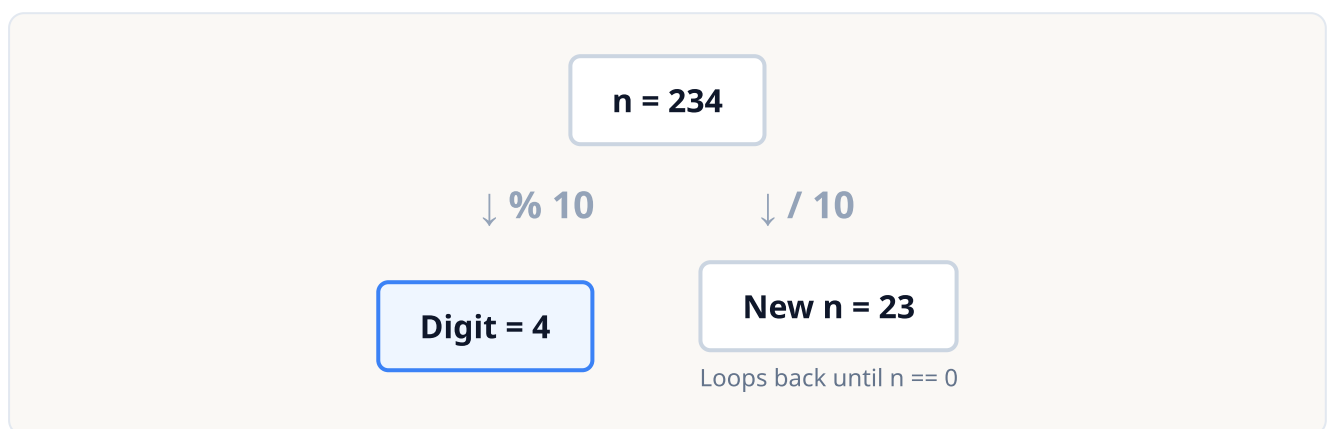
THE MODULO AND DIVISION TRICK

To process numbers digit by digit in Java:

1. $\text{digit} = n \% 10$ (Extracts the last digit)

2. $n = n / 10$ (Removes the last digit)

3. VISUALIZE THE EXTRACTION



4. STEP-BY-STEP APPROACH

1. Initialize $\text{product} = 1$ and $\text{sum} = 0$.
2. While $n > 0$:
 - Extract the last digit using $n \% 10$.
 - Multiply the digit to product.
 - Add the digit to sum.

5. JAVA SOLUTION

```
class Solution {
    public int subtractProductAndSum(int n) {
        int product = 1;
        int sum = 0;

        while (n > 0) {
            int digit = n % 10;    // Extract last digit
            product *= digit;      // Update product
            sum += digit;          // Update sum
            n /= 10;              // Remove last digit
        }

        return product - sum;
    }
}
```

6. DRY RUN (N = 234)

n	Condition (n > 0)	digit (n % 10)	product	sum	Next n (n / 10)
234	True	4	1 * 4 = 4	0 + 4 = 4	23
23	True	3	4 * 3 = 12	4 + 3 = 7	2
2	True	2	12 * 2 = 24	7 + 2 = 9	0
0	False (Loop ends)	-	24	9	-

Final Return: 24 - 9 = 15

7. COMPLEXITY

Time: $O(\log_{10} n)$

Space: $O(1)$

The loop runs once for each digit in the number. The number of digits in n is proportional to $\log_{10}(n)$. Space is $O(1)$ as we only use primitive integer variables.

Problem 1342 — Number of Steps to Reduce a Number to Zero

EASY

PATTERN: SIMULATION

1. UNDERSTAND THE PROBLEM

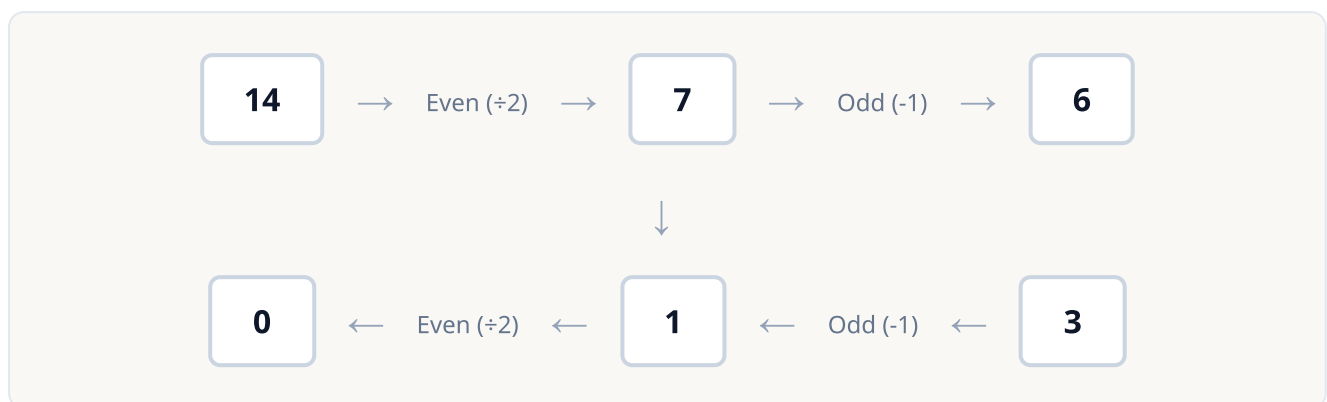
Given: An integer num.

Rules:

- If the number is even, divide it by 2.
- If the number is odd, subtract 1 from it.

Find: The total number of steps required to reach 0.

2. VISUALIZE THE STATE TRANSITION



3. STEP-BY-STEP APPROACH

1. Keep a counter `steps = 0`.
2. While `num > 0`, check if it's even or odd using modulo (`num % 2 == 0`).
3. If even, update `num = num / 2`.
4. If odd, update `num = num - 1`.
5. Increment the steps counter.
6. Once the loop finishes, return steps.

4. JAVA SOLUTION

```
class Solution {
    public int numberOfSteps(int num) {
        int steps = 0;

        while (num > 0) {
            if (num % 2 == 0) {
                num /= 2;    // Even: divide by 2
            } else {
                num -= 1;    // Odd: subtract 1
            }
            steps++;        // Count this step
        }

        return steps;
    }
}
```

5. DRY RUN (NUM = 14)

num before step	Condition	Action	num after step	steps counter
14	Even (14 % 2 == 0)	Divide by 2	7	1
7	Odd (7 % 2 != 0)	Subtract 1	6	2
6	Even (6 % 2 == 0)	Divide by 2	3	3
3	Odd (3 % 2 != 0)	Subtract 1	2	4
2	Even (2 % 2 == 0)	Divide by 2	1	5
1	Odd (1 % 2 != 0)	Subtract 1	0	6

Number is now 0. Loop terminates. Return **6**.

6. COMPLEXITY

Time: $O(\log n)$

Space: $O(1)$

At worst, the number is odd, we subtract 1, making it even. Then we divide by 2, halving it. Halving operations result in a logarithmic time complexity.

Problem 412 — Fizz Buzz

EASY

PATTERN: SIMULATION / STRING

1. UNDERSTAND THE PROBLEM

Given: An integer n .

Find: A string array (or List) containing numbers from 1 to n , but with replacements:

- Divisible by 3 and 5 $\rightarrow$ "FizzBuzz"
- Divisible by 3 only $\rightarrow$ "Fizz"
- Divisible by 5 only $\rightarrow$ "Buzz"
- None of the above $\rightarrow$ The number as a string (e.g., "1")

2. KEY IDEA

CONDITION ORDERING IS CRITICAL

A number divisible by both 3 and 5 is actually divisible by 15. You **MUST** check the $15 == 0$ condition **first**. If you check $3 == 0$ first, "15" will be evaluated as "Fizz" instead of "FizzBuzz".

3. JAVA SOLUTION

```
class Solution {
    public List<String> fizzBuzz(int n) {
        List<String> result = new ArrayList<>();

        for (int i = 1; i <= n; i++) {
            // Check divisible by both 3 and 5 first
            if (i % 15 == 0) {
                result.add("FizzBuzz");
            }
            // Check divisible by 3
            else if (i % 3 == 0) {
                result.add("Fizz");
            }
            // Check divisible by 5
            else if (i % 5 == 0) {
                result.add("Buzz");
            }
            // Not divisible by 3 or 5
            else {
                result.add(String.valueOf(i));
            }
        }
    }
}
```

4. DRY RUN (N = 5)

i	i % 15 == 0	i % 3 == 0	i % 5 == 0	Action	List State
1	False	False	False	Add "1"	["1"]
2	False	False	False	Add "2"	["1", "2"]
3	False	True	-	Add "Fizz"	["1", "2", "Fizz"]
4	False	False	False	Add "4"	["1", "2", "Fizz", "4"]
5	False	False	True	Add "Buzz"	["1", "2", "Fizz", "4", "Buzz"]

5. COMMON MISTAKES

- **Wrong Order:** Checking `i % 3` before `i % 15`.
- **Using Multiple `if` without `else`:** If you don't use `else if`, a number like 15 might trigger both the "FizzBuzz" block AND the "Fizz" block if constructed incorrectly.

6. COMPLEXITY

Time: $O(n)$

Space: $O(n)$ (for output array)

We loop from 1 to n exactly once, doing $O(1)$ work per iteration. We create a list of size n to store the result.

Problem 2520 — Count the Digits That Divide a Number

EASY

PATTERN: DIGIT EXTRACTION

1. UNDERSTAND THE PROBLEM

Given: An integer num consisting of non-zero digits.

Find: The count of digits in num that can completely divide num without a remainder.

Example: num = 121

Digit 1: $121 \% 1 == 0$ (Yes)

Digit 2: $121 \% 2 != 0$ (No)

Digit 1: $121 \% 1 == 0$ (Yes)

Total count = 2

2. KEY IDEA

PRESERVE THE ORIGINAL NUMBER

We need to extract digits one by one. But we also need the *original* number to check divisibility ($\text{num} \% \text{digit} == 0$). Therefore, we must copy num into a temporary variable (temp) and extract digits from temp, leaving num intact.

3. JAVA SOLUTION

```
class Solution {
    public int countDigits(int num) {
        int count = 0;
        int temp = num; // Use temp to extract digits

        while (temp > 0) {
            int digit = temp % 10; // Extract last digit

            // Check if digit divides the original num
            if (num % digit == 0) {
                count++;
            }

            temp /= 10; // Remove last digit
        }

        return count;
    }
}
```

4. DRY RUN (NUM = 124)

num	temp	digit (temp % 10)	Check (num % digit == 0)	count	Next temp (temp / 10)
124	124	4	124 % 4 == 0 (True)	1	12
124	12	2	124 % 2 == 0 (True)	2	1
124	1	1	124 % 1 == 0 (True)	3	0

Return **3**.

5. EDGE CASES ADDRESSED

- The problem guarantees that all digits are non-zero, so we never have to worry about a `ArithmeticException: / by zero` when doing `num % digit`.

6. COMPLEXITY

Time: $O(\log_{10} n)$

Space: $O(1)$

The time complexity is proportional to the number of digits, which is logarithmic base 10.

Problem 2652 — Sum Multiples

EASY

PATTERN: MATH

1. UNDERSTAND THE PROBLEM

Given: A positive integer n .

Find: The sum of all integers in the range $[1, n]$ that are divisible by 3, 5, or 7.

2. APPROACH

This is a straightforward iteration problem:

1. Create a sum variable initialized to 0.
2. Use a loop to check every number i from 1 up to n .
3. If $i \% 3 == 0$ OR $i \% 5 == 0$ OR $i \% 7 == 0$, add i to sum.
4. Return sum.

3. JAVA SOLUTION

```
class Solution {  
    public int sumOfMultiples(int n) {  
        int sum = 0;  
  
        for (int i = 1; i <= n; i++) {  
            // Check divisibility by 3, 5, or 7 using Logical OR (||)  
            if (i % 3 == 0 || i % 5 == 0 || i % 7 == 0) {  
                sum += i;  
            }  
        }  
  
        return sum;  
    }  
}
```

4. COMPLEXITY

Time: $O(n)$

Space: $O(1)$

We examine every number up to n exactly once.

Problem 9 — Palindrome Number

EASY

PATTERN: DIGIT EXTRACTION

1. UNDERSTAND THE PROBLEM

Given: An integer x .

Find: Return `true` if x reads the same forward and backward, otherwise `false`.

Constraint Idea: Could you solve it without converting the integer to a string?

2. KEY IDEA

REVERSE HALF THE NUMBER

Instead of reversing the whole number (which can cause integer overflow), reverse only the *second half* of the digits. If the original first half matches the reversed second half, it's a palindrome.

3. VISUALIZE REVERSING HALF



Stop because x (12) is no longer $> rev$ (12). They match!

4. EDGE CASES

- Negative numbers are NEVER palindromes (e.g., -121 backward is 121-).
- Numbers ending in 0 (except 0 itself) are NEVER palindromes (e.g., 10 backward is 01, which is not 10).

5. JAVA SOLUTION

```
class Solution {
    public boolean isPalindrome(int x) {
        // Edge cases: Negative numbers or trailing zero (except zero itself)
        if (x < 0 || (x % 10 == 0 && x != 0)) {
            return false;
        }

        int reversed = 0;
        // Stop when the original number is less than or equal to the reversed
        half
        while (x > reversed) {
            reversed = reversed * 10 + x % 10;
            x /= 10;
        }

        // For even digits: x == reversed (e.g. 1221 -> x=12, rev=12)
        // For odd digits: x == reversed/10 (e.g. 12321 -> x=12, rev=123)
        return x == reversed || x == reversed / 10;
    }
}
```

6. DRY RUN (ODD LENGTH: X = 12321)

x	Condition (x > reversed)	reversed calculation	new x
12321	12321 > 0 (True)	0 * 10 + 1 = 1	1232
1232	1232 > 1 (True)	1 * 10 + 2 = 12	123
123	123 > 12 (True)	12 * 10 + 3 = 123	12
12	12 > 123 (False)	Loop ends	-

Evaluation: x (12) == reversed (123) is False, BUT x (12) == reversed / 10 (12) is True. Return **true**.

7. COMPLEXITY

Time: $O(\log_{10} n)$

Space: $O(1)$

We only traverse half of the digits, ensuring efficiency and no chance of overflow.

Problem 258 — Add Digits

EASY

PATTERN: MATH

1. UNDERSTAND THE PROBLEM

Given: An integer num.

Find: Repeatedly add all its digits until the result has only one digit, and return it.

Example: num = 38

Step 1: $3 + 8 = 11$

Step 2: $1 + 1 = 2$

Result: 2

2. KEY IDEA (O(1) APPROACH)

THE DIGITAL ROOT

This is a classic math problem known as the "Digital Root". The sum of digits of a number to a single digit follows a predictable repeating pattern based on modulo 9.

Formula: $1 + (\text{num} - 1) \% 9$.

If num is a multiple of 9, its digital root is 9.

3. JAVA SOLUTION

```
class Solution {
    public int addDigits(int num) {
        // Base case
        if (num == 0) {
            return 0;
        }
        // If perfectly divisible by 9, the digital root is 9
        if (num % 9 == 0) {
            return 9;
        }
        // Otherwise, it's the remainder when divided by 9
        return num % 9;
    }
}
```

4. COMPLEXITY

Problem 509 — Fibonacci Number

EASY

PATTERN: DYNAMIC PROGRAMMING / SIMULATION

1. UNDERSTAND THE PROBLEM

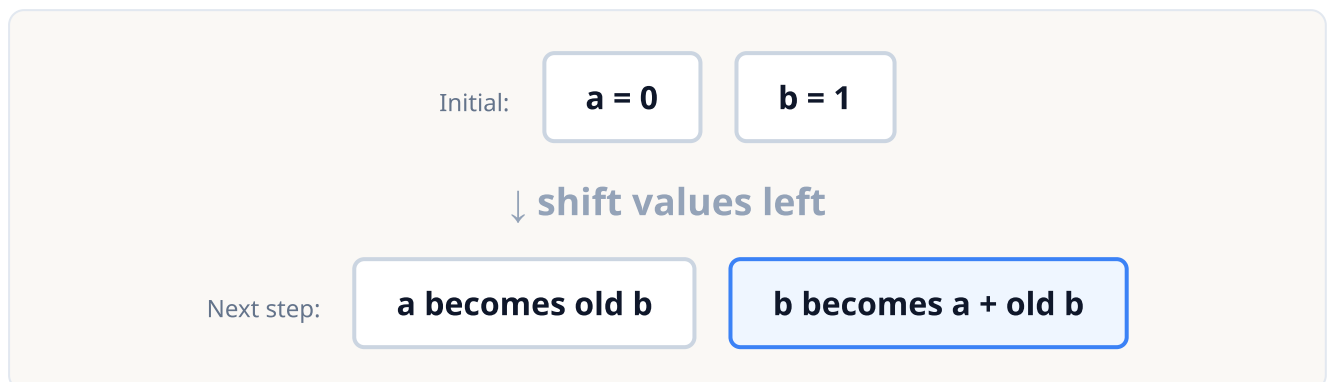
Given: An integer n .

Find: The n -th Fibonacci number.

Rules:

- $F(0) = 0$
- $F(1) = 1$
- $F(n) = F(n - 1) + F(n - 2)$, for $n > 1$

2. VISUALIZE THE STATE SHIFT



3. WHY NOT RECURSION?

Simple recursion $\text{fib}(n-1) + \text{fib}(n-2)$ recalculates the same numbers over and over, leading to $O(2^n)$ time complexity. Using two variables updates the state iteratively in $O(n)$ time and $O(1)$ space, which is highly efficient.

4. JAVA SOLUTION

```
class Solution {
    public int fib(int n) {
        // Handle base cases immediately
        if (n <= 1) {
            return n;
        }

        int a = 0;
        int b = 1;

        // Build from bottom up
        for (int i = 2; i <= n; i++) {
            int next = a + b;
            a = b;        // Shift a
            b = next;     // Shift b
        }

        return b;
    }
}
```

5. DRY RUN (N = 4)

i	next (a + b)	new a (was b)	new b (was next)
-	-	0	1
2	0 + 1 = 1	1	1
3	1 + 1 = 2	1	2
4	1 + 2 = 3	2	3

Loop ends. Return b which is **3**.

6. COMPLEXITY

Time: $O(n)$

Space: $O(1)$

We loop $n-1$ times doing constant work. We only store two variables, ignoring recursion stack space.

Problem 1137 — N-th Tribonacci Number

EASY

PATTERN: DYNAMIC PROGRAMMING

1. UNDERSTAND THE PROBLEM

Given: An integer n .

Find: The n -th Tribonacci number.

Rules:

- $T(0) = 0$
- $T(1) = 1$
- $T(2) = 1$
- $T(n) = T(n - 1) + T(n - 2) + T(n - 3)$, for $n > 2$

2. KEY IDEA

THE THREE-VARIABLE WINDOW

This is exactly like the Fibonacci problem, but instead of maintaining a window of 2 variables, we maintain a sliding window of 3 variables (a , b , c).

3. JAVA SOLUTION

```
class Solution {
    public int tribonacci(int n) {
        // Base cases
        if (n == 0) return 0;
        if (n == 1 || n == 2) return 1;

        int a = 0;
        int b = 1;
        int c = 1;

        // Calculate iteratively
        for (int i = 3; i <= n; i++) {
            int next = a + b + c;
            a = b;
            b = c;
            c = next;
        }

        return c;
    }
}
```

4. DRY RUN (N = 4)

i	next (a + b + c)	new a (b)	new b (c)	new c (next)
-	-	0	1	1
3	$0 + 1 + 1 = 2$	1	1	2
4	$1 + 1 + 2 = 4$	1	2	4

Loop ends. Return c which is **4**.

5. COMPLEXITY

Time: $O(n)$

Space: $O(1)$

Like Fibonacci, calculating state forwards avoids deep recursive call stacks. Space is $O(1)$ because we reuse the exact same three integer variables regardless of how large n is.

Problem 231 — Power of Two

EASY

PATTERN: MATH / BITWISE

1. UNDERSTAND THE PROBLEM

Given: An integer n .

Find: Return true if n is a power of two, otherwise false.

2. TWO WAYS TO THINK ABOUT IT

1. **Math Approach (Beginner):** Keep dividing n by 2 as long as it's perfectly divisible. If you end up with 1, it was a power of 2.
2. **Bitwise Trick (Advanced but elegant):** A power of 2 in binary is a 1 followed by all 0s (e.g., 8 is 1000). Subtracting 1 flips all bits (e.g., 7 is 0111). Therefore, $n \& (n - 1) == 0$ only if n is a power of 2.

We will show the elegant bitwise approach as it is the standard $O(1)$ interview solution.

3. JAVA SOLUTION

```
class Solution {  
    public boolean isPowerOfTwo(int n) {  
        // Powers of 2 must be positive  
        if (n <= 0) {  
            return false;  
        }  
  
        // Bitwise AND trick to check if exactly one bit is set  
        return (n & (n - 1)) == 0;  
    }  
}
```

4. COMPLEXITY

Time: $O(1)$

Space: $O(1)$

Bitwise operations evaluate in constant time at the CPU level.

Problem 326 — Power of Three

EASY

PATTERN: MATH

1. UNDERSTAND THE PROBLEM

Given: An integer n .

Find: Return true if it is a power of three, otherwise false.

Note: We cannot use the bitwise trick here because base-3 doesn't map cleanly to binary bits like base-2 does.

2. APPROACH (SIMULATION)

1. If $n \leq 0$, return false.
2. While n is perfectly divisible by 3 ($n \% 3 == 0$), divide it by 3 ($n /= 3$).
3. When the loop stops, if $n == 1$, it was a power of 3. If it's anything else, it wasn't.

3. JAVA SOLUTION

```
class Solution {
    public boolean isPowerOfThree(int n) {
        if (n <= 0) return false;

        // Keep reducing by a factor of 3
        while (n % 3 == 0) {
            n /= 3;
        }

        // True if we reduced perfectly to 1
        return n == 1;
    }
}
```

4. COMPLEXITY

Time: $O(\log_3 n)$

Space: $O(1)$

Problem 1523 — Count Odd Numbers in an Interval Range

EASY

PATTERN: MATH

1. UNDERSTAND THE PROBLEM

Given: Two non-negative integers low and high.

Find: The count of odd numbers between low and high (inclusive).

Example: low = 3, high = 7 → Odds: 3, 5, 7. Count = 3.

2. KEY IDEA

MATH OVER LOOPS

A for loop from low to high takes $O(N)$ time. If the range is 1 billion, a loop will Time Out. Instead, use math.

Half the numbers in any range are odd: $(\text{high} - \text{low}) / 2$.

If either boundary is odd, there's one extra odd number in the range!

3. JAVA SOLUTION

```
class Solution {
    public int countOdds(int low, int high) {
        // Base count: half of the numbers between high and low are odd
        int count = (high - low) / 2;

        // If either endpoint is odd, add 1 to compensate
        if (low % 2 != 0 || high % 2 != 0) {
            count++;
        }

        return count;
    }
}
```

4. COMPLEXITY

Time: $O(1)$

Space: $O(1)$

15-Problem Quick Revision

#	Problem	Core Pattern	Time	Key Takeaway
2235	Add Two Integers	Math	O(1)	Basic math operators
2469	Convert Temperature	Math	O(1)	Multiple math returns
2413	Smallest Even Multiple	Math	O(1)	Check divisibility by 2
1281	Subtract Prod and Sum	Digit Extract	O(log n)	% 10 and / 10 loop
1342	Reduce to Zero	Simulation	O(log n)	Halving shrinks fast
412	Fizz Buzz	Simulation	O(n)	Order of if-checks matters
2520	Count Digits That Divide	Digit Extract	O(log n)	Save original before loop
2652	Sum Multiples	Math	O(n)	Logical OR inside loop
9	Palindrome Number	Digit Extract	O(log n)	Reverse half to avoid overflow
258	Add Digits	Math	O(1)	Digital root via mod 9
509	Fibonacci Number	DP (Sim)	O(n)	Two vars for state shifting
1137	Tribonacci Number	DP (Sim)	O(n)	Three vars for state shifting
231	Power of Two	Bitwise	O(1)	$n \ \& \ (n-1) == 0$
326	Power of Three	Simulation	O(log n)	Repeated division by 3
1523	Count Odd Numbers	Math	O(1)	Math beats loops for ranges

PATTERN REVISION